III.4. Reprezentări în virgulă fixă

Reprezentări numerice: probleme

- reprezentarea semnului
  - nu există simbol special, doar cele pentru cifre
- virgula
  - trebuie cunoscută în orice moment poziția sa
- operații aritmetice
  - implementare cât mai eficientă
  - nu este posibil pentru toate operațiile simultan
  - trebuie decis pe care le optimizăm

Codificări în virgulă fixă

- semnul - este folosit unul dintre biți
- virgula
  - are întotdeauna aceeași poziție în șirul de cifre
    - deci poziția nu mai trebuie memorată
- operații implementate eficient
  - adunare, scădere
- codificările - pe **n**+**m** biţi (**n**≥1, **m**≥0)
  - **m**=0 - numere întregi
  - **n**=1 - numere subunitare

Codificări redundante

- codificare redundantă
  - există cel puțin un număr cu două reprezentări diferite
  - probleme la operațiile aritmetice
- codificările folosite în practică
  - reprezentarea numerelor pozitive - la fel ca la numere fără semn; diferențe - numere negative
  - unele prezintă două reprezentări pentru 0

Reprezentarea prin modul şi semn

- notație: A+S

$$val_{A+S}^{n,m}(a_{n-1}a_{n-2}...a_1a_0a_{-1}...a_{-m})=$$

$$=\begin{cases} a_{n-2}\times 2^{n-2}+...+a_{-m}\times 2^{-m} & \text{dacă } a_{n-1}=0 \\ -(a_{n-2}\times 2^{n-2}+...+a_{-m}\times 2^{-m}) & \text{dacă } a_{n-1}=1 \end{cases}$$

- similar cu scrierea în baza 2
  - bitul cel mai din stânga reprezintă semnul
  - virgula este implicită

Modul şi semn - limite

- pe n+m biți sunt 2^{n+m} reprezentări diferite
  - dar numai $2^{n+m}-1$ numere diferite
  - redundantă: $val_{A+S}^{n,m}(00...0)=val_{A+S}^{n,m}(10...0)=0$
- valorile extreme reprezentabile

$$max_{A+S}^{n,m}=val_{A+S}^{n,m}(01...1)=2^{n-1}-2^{-m}$$

$$min_{A+S}^{n,m}=val_{A+S}^{n,m}(11...1)=-\left(2^{n-1}-2^{-m}\right)$$

  - deci numerele reprezentabile sunt în intervalul $[-(2^{n-1}-2^{-m}); +(2^{n-1}-2^{-m})]$

Modul şi semn - precizie

- numerele reprezentabile exact încep cu min=$-(2^{n-1}-2^{-m})$
  - și continuă cu pasul 2^{-m}
- celelalte numere din interval - aproximare
  - eroarea - cel mult 2^{-m}
- deci precizia reprezentării este 2^{-m}
- pentru n+m fixat
  - numere mai mari = precizie mai slabă și invers

Exemple (1)

$$val_{A+S}^{8,0}(00110011)=2^5+2^4+2^1+2^0=51$$

$$val_{A+S}^{6,2}(00110011)=2^3+2^2+2^{-1}+2^{-2}=12,75$$

sau

$$val_{A+S}^{6,2}(00110011)=val_{A+S}^{8,0}(00110011):2^2=51:4=12,75$$

$$val_{A+S}^{4,4}(00110011)=2^1+2^0+2^{-3}+2^{-4}=3,1875$$

sau

$$val_{A+S}^{4,4}(00110011)=val_{A+S}^{8,0}(00110011):2^4=51:16=3,1875$$

Exemple (2)

$$val_{A+S}^{8,0}(10110011)=-(2^5+2^4+2^1+2^0)=-51$$

$$val_{A+S}^{4,4}(10110011)=-(2^1+2^0+2^{-3}+2^{-4})=-3,1875$$

sau

$$val_{A+S}^{4,4}(10110011)=val_{A+S}^{8,0}(10110011):2^4=-51:16=-3,1875$$

$$min_{A+S}^{8,0}=val_{A+S}^{8,0}(11111111)=-127$$

$$min_{A+S}^{4,4}=val_{A+S}^{4,4}(11111111)=-7,9375$$

sau

$$min_{A+S}^{4,4}=min_{A+S}^{8,0}:2^4=-127:16=-7,9375$$

Exemple (3)

$\max_{A+S}^{8,0}=val_{A+S}^{8,0}(01111111)=127$

$\max_{A+S}^{4,4}=val_{A+S}^{4,4}(01111111)=7{,}9375$

sau

$\max_{A+S}^{4,4}=\max_{A+S}^{8,0}:2^4=127:16=7{,}9375$

- intervale reprezentabile
  - $A+S^{8,0}$: [-127; 127] → 255 numere, din 1 în 1
  - $A+S^{4,4}$: [-7,9375; 7,9375] → 255 numere, din 0,0625 în 0,0625 (=1:16)

Operații în A+S

- adunare/scădere
  - stabilirea semnului rezultatului (comparaţie)
  - aplicarea algoritmilor cunoscuți
- înmulţire/împărţire
  - similar algoritmilor cunoscuți
- în general mai complex decât ne-am dori
  - nu putem utiliza pur și simplu un sumator "clasic" pentru adunare

Reprezentarea în complement față de 1

- notație: C_1

$$val_{C_1}^{n,m}(a_{n-1}a_{n-2}...a_1a_0a_{-1}...a_{-m})=$$

$$=\begin{cases} a_{n-2}\times 2^{n-2}+...+a_{-m}\times 2^{-m} & \text{dacă } a_{n-1}=0 \\ (a_{n-2}\times 2^{n-2}+...+a_{-m}\times 2^{-m})-(2^{n-1}-2^{-m}) & \text{dacă } a_{n-1}=1 \end{cases}$$

- temă: demonstrați că valoarea este negativă pentru $a_{n-1} = 1$
  - deci a_{n-1} reprezintă semnul

Complement față de 1 - limite

- pe n+m biți sunt 2^{n+m} reprezentări diferite
  - dar numai $2^{n+m}-1$ numere diferite
  - redundantă: $val_{C_1}^{n,m}(00...0)=val_{C_1}^{n,m}(11...1)=0$
- valorile extreme reprezentabile

$$max_{C_1}^{n,m}=val_{C_1}^{n,m}(01...1)=2^{n-1}-2^{-m}$$

$$min_{C_1}^{n,m}=val_{C_1}^{n,m}(10...0)=-\left(2^{n-1}-2^{-m}\right)$$

  - deci numerele reprezentabile sunt în intervalul $[-(2^{n-1}-2^{-m}); +(2^{n-1}-2^{-m})]$

Complement față de 1 - precizie

- numerele reprezentabile exact încep cu min=$-(2^{n-1}-2^{-m})$
  - și continuă cu pasul 2^{-m}
- celelalte numere din interval - aproximare
  - eroarea - cel mult 2^{-m}
- deci precizia reprezentării este 2^{-m}
- pentru n+m fixat
  - numere mai mari = precizie mai slabă și invers

Complementare

- reprezentările pentru numerele pozitive - ușor de determinat
- pentru numere negative - mai greu
- există o relație între reprezentarea numărului q și cea a numărului $-q$?
- da: reprezentarea lui $-q$ se obține negând toți biții din reprezentarea lui q
  - operație comutativă - valabilă și pentru $q < 0$

Exemple (1)

$$val_{C_1}^{8,0}(00110011)=2^5+2^4+2^1+2^0=51$$

$$val_{C_1}^{6,2}(00110011)=2^3+2^2+2^{-1}+2^{-2}=12,75$$

sau

$$val_{C_1}^{6,2}(00110011)=val_{C_1}^{8,0}(00110011):2^2=51:4=12,75$$

$$val_{C_1}^{4,4}(00110011)=2^1+2^0+2^{-3}+2^{-4}=3,1875$$

sau

$$val_{C_1}^{4,4}(00110011)=val_{C_1}^{8,0}(00110011):2^4=51:16=3,1875$$

Exemple (2)

$$val_{C_1}^{8,0}(11001100)=(2^6+2^3+2^2)-(2^7-2^0)=-51$$

$$val_{C_1}^{4,4}(11001100)=(2^2+2^{-1}+2^{-2})-(2^3-2^{-4})=-3,1875$$

sau

$$val_{C_1}^{4,4}(11001100)=val_{C_1}^{8,0}(11001100):2^4=-51:16=-3,1875$$

$$min_{C_1}^{8,0}=val_{C_1}^{8,0}(10000000)=0-(2^7-2^0)=-127$$

$$min_{C_1}^{4,4}=val_{C_1}^{4,4}(10000000)=0-(2^3-2^{-4})=-7,9375$$

sau

$$min_{C_1}^{4,4}=min_{C_1}^{8,0}:2^4=-127:16=-7,9375$$

Exemple (3)

$$\max_{C_1}^{8,0} = \mathrm{val}_{C_1}^{8,0}(01111111) = 127$$

$$\max_{C_1}^{4,4} = \mathrm{val}_{C_1}^{4,4}(01111111) = 7{,}9375$$

sau

$$\max_{C_1}^{4,4} = \max_{C_1}^{8,0} : 2^4 = 127:16 = 7{,}9375$$

- intervale reprezentabile
  - $C_1^{8,0}$: [-127; 127] $\rightarrow$ 255 numere, din 1 în 1
  - $C_1^{4,4}$: [-7,9375; 7,9375] $\rightarrow$ 255 numere, din 0,0625 în 0,0625 (=1:16)

Operații în C_1

- putem aduna două numere în C_1 cu ajutorul unui sumator "clasic"?
- da, dar în doi pași
  - în al doilea pas, la rezultat se adună transportul obținut la primul pas
  - deci trebuie două sumatoare pentru o adunare
- scădere: adunăm descăzutul cu simetricul scăzătorului

Reprezentarea în complement faţă de 2

- cerințe
  - reprezentare neredundantă
    - o singură reprezentare pentru 0
  - adunarea a două numere se poate realiza cu un singur sumator
    - la fel ca la numere fără semn
    - beneficiu - o singură operație de adunare implementată în procesor pentru tipuri de date cu semn și fără semn

Complement faţă de 2

- notație: C_2

$$val_{C_2}^{n,m}(a_{n-1}a_{n-2}...a_1a_0a_{-1}...a_{-m})=$$

$$=\begin{cases} a_{n-2}\times 2^{n-2}+...+a_{-m}\times 2^{-m} & \text{dacă } a_{n-1}=0 \\ (a_{n-2}\times 2^{n-2}+...+a_{-m}\times 2^{-m})-2^{n-1} & \text{dacă } a_{n-1}=1 \end{cases}$$

- temă: demonstrați că valoarea este negativă pentru $a_{n-1} = 1$
  - deci a_{n-1} reprezintă semnul

Complement față de 2 - limite

- pe n+m biți sunt 2^{n+m} reprezentări diferite
  - și 2^{n+m} numere diferite
  - 00...0 - singura reprezentare pentru 0
- valorile extreme reprezentabile

$$\max_{C_2}^{n,m} = val_{C_2}^{n,m}(01...1) = 2^{n-1} - 2^{-m}$$

$$\min_{C_2}^{n,m} = val_{C_2}^{n,m}(10...0) = -2^{n-1}$$

  - deci numerele reprezentabile sunt în intervalul $[-2^{n-1}; +(2^{n-1}-2^{-m})]$ - asimetric

Complement față de 2 - precizie

- numerele reprezentabile exact încep cu min=-2^{n-1}
  - și continuă cu pasul 2^{-m}
- celelalte numere din interval - aproximare
  - eroarea - cel mult 2^{-m}
- deci precizia reprezentării este 2^{-m}
- pentru n+m fixat
  - numere mai mari = precizie mai slabă și invers

Complementare (1)

- există o relație între reprezentarea numărului *q* și cea a numărului -*q*?
- da: reprezentarea lui -*q* este complementul faţă de 2 al reprezentării lui *q*
  - se neagă toți biții și se adună 0...01
  - la fel ca la C_1, operația este comutativă - se poate aplica indiferent de semnul *q*

Complementare (2)

- exemplu

  $q = 77$ are reprezentarea 01001101 în $C_2^{8,0}$

  $-q = -77$ are reprezentarea 10110010 + 00000001 = 10110011

- temă
  - reprezentarea în C_2 pe N biţi a numărului întreg negativ q este de fapt reprezentarea pe N biţi a numărului $q + 2^N = 2^N - |q|$

Exemple (1)

$$val_{C_2}^{8,0}(00110011)=2^5+2^4+2^1+2^0=51$$

$$val_{C_2}^{6,2}(00110011)=2^3+2^2+2^{-1}+2^{-2}=12,75$$

sau

$$val_{C_2}^{6,2}(00110011)=val_{C_2}^{8,0}(00110011):2^2=51:4=12,75$$

$$val_{C_2}^{4,4}(00110011)=2^1+2^0+2^{-3}+2^{-4}=3,1875$$

sau

$$val_{C_2}^{4,4}(00110011)=val_{C_2}^{8,0}(00110011):2^4=51:16=3,1875$$

Exemple (2)

$$val_{C_2}^{8,0}(11001101)=(2^6+2^3+2^2)-(2^7-2^0)=-51$$

$$val_{C_2}^{4,4}(11001101)=(2^2+2^{-1}+2^{-2})-(2^3-2^{-4})=-3,1875$$

sau

$$val_{C_2}^{4,4}(11001101)=val_{C_2}^{8,0}(11001101):2^4=-51:16=-3,1875$$

$$min_{C_2}^{8,0}=val_{C_2}^{8,0}(10000000)=0-2^7=-128$$

$$min_{C_2}^{4,4}=val_{C_2}^{4,4}(10000000)=0-2^3=8$$

sau

$$min_{C_2}^{4,4}=min_{C_2}^{8,0}:2^4=-128:16=-8$$

Exemple (3)

$$\max_{C_2}^{8,0} = \mathrm{val}_{C_2}^{8,0}(01111111) = 127$$

$$\max_{C_2}^{4,4} = \mathrm{val}_{C_2}^{4,4}(01111111) = 7{,}9375$$

sau

$$\max_{C_2}^{4,4} = \max_{C_2}^{8,0} : 2^4 = 127:16 = 7{,}9375$$

- intervale reprezentabile
  - $C_2^{8,0}$: [-128; 127] → 256 numere, din 1 în 1
  - $C_2^{4,4}$: [-8; 7,9375] → 256 numere, din 0,0625 în 0,0625 (=1:16)

Concluzii

- C_2 este reprezentarea utilizată cel mai des
  - neredundantă
  - adunarea/scăderea - implementate la fel ca la numere fără semn
- în practică - tipuri de date întregi din limbajele de programare
  - caz particular (m=0)
  - pentru numere reale se utilizează reprezentările în virgulă mobilă

III.5. Depăşiri pentru operaţii cu reprezentări în virgulă fixă

Depășiri

- nu sunt suficienți biți la partea întreagă pentru numărul de reprezentat
  - numărul este în afara intervalului reprezentabil
- problemă
  - având două numere reprezentabile, rezultatul unei operații efectuate asupra lor poate să nu fie reprezentabil - *depășire*
  - când se întâmplă și cum detectăm așa ceva?

Trecerea la reprezentări mai lungi

- având un număr reprezentat pe *n* biți, cum obținem reprezentarea sa pe *n*+*k* biți?
  - adăugare de cifre nesemnificative la partea întreagă; partea fracționară nu e afectată
- A+S: se adaugă *k* zerouri imediat la dreapta cifrei semn
- C_1, C_2: se repetă cifra semn de *k* ori imediat la dreapta cifrei semn

Exemple

	număr	
codificare	51	-51
$A+S^{8,0}$	00110011	10110011
$A+S^{16,0}$	0000000000110011	1000000000110011
$C_1^{8,0}$	00110011	11001100
$C_1^{16,0}$	0000000000110011	1111111111001100
$C_2^{8,0}$	00110011	11001101
$C_2^{16,0}$	0000000000110011	1111111111001101

Trecerea la reprezentări mai scurte

- folosim aceste rezultate pentru a răspunde la problema inversă
- având un număr reprezentat pe *n* biți, poate fi reprezentat pe *n*-*k* biți?
  - da, dacă și numai dacă cei *k* biți de la dreapta celui de semn au valorile ca mai înainte
  - în acest caz, ei pot fi eliminați din reprezentare

Operaţii în C_2

- în continuare vom discuta doar cazul C_2
  - reprezentarea cel mai des folosită
- restricții impuse de calculator asupra operațiilor cu reprezentări
  - termenii sumei şi rezultatul se reprezintă pe acelaşi număr de biţi
  - termenii înmulțirii se reprezintă pe acelaşi număr de biţi, iar rezultatul pe număr dublu de biți

Definiția depășirii

- fie o reprezentare dată şi *op* o operaţie cu numere
- pe n+m biți, numerele reprezentabile sunt în intervalul [min; max]
- fie două numere a, b $\in$ [min; max]
- operaţia *op* aplicată numerelor a şi b produce depăşire dacă

$$a \; op \; b \notin [min; max]$$

Exemple (1)

- în continuare vom folosi reprezentarea C_2 cu n=4, m=0

1111 + 1111 = 11110 → 1110

– bitul "suplimentar" este ignorat (doar 4 biți)

– de fapt este bitul de transport

$\mathrm{val}_{C_2}^{4,0}(1111)=-1$

$\mathrm{val}_{C_2}^{4,0}(1110)=-2$

– rezultatul este corect - nu avem depășire

Exemple (2)

0111 + 0111 = 1110 → 1110

– nu avem bit "suplimentar" (transportul este 0)

$\mathrm{val}_{C_2}^{4,0}(0111)=7$

$\mathrm{val}_{C_2}^{4,0}(1110)=-2$

– rezultat incorect - se produce depășire

- concluzie - bitul de transport nu oferă informații privind depășirea
  – trebuie căutată altă formă de detecție

Condiţia de depăşire

- nu putem folosi direct definiția depășirii
  - numerele nu sunt disponibile
- constatare
  - depăşire se poate produce la adunare doar când ambii operanzi au acelaşi semn
    - iar reprezentarea rezultatului indică semn opus
- temă: nu se poate produce depășire la adunarea a două numere de semn opus

Suma algebrică în C_2 (1)

- Teorema 1

  dacă numerele a și b sunt reprezentabile în $C_2^{n,m}$, atunci a ± b sunt reprezentabile în $C_2^{n+1,m}$

- Lemă

dacă $a=val_{C_2}^{n+1,m}(\alpha_n\alpha_{n-1}...\alpha_1\alpha_0\alpha_{-1}...\alpha_{-m})$ și $\alpha_n=\alpha_{n-1}$

atunci $a=val_{C_2}^{n,m}(\alpha_{n-1}...\alpha_1\alpha_0\alpha_{-1}...\alpha_{-m})$

Suma algebrică în C_2 (2)

- fie reprezentările

$\alpha = \alpha_{n-1}\alpha_{n-2}\ldots\alpha_1\alpha_0\alpha_{-1}\ldots\alpha_{-m}$

$\beta = \beta_{n-1}\beta_{n-2}\ldots\beta_1\beta_0\beta_{-1}\ldots\beta_{-m}$

- definim suma lor formală $\gamma = \alpha + \beta$ ca

$\gamma = \gamma_n\gamma_{n-1}\gamma_{n-2}\ldots\gamma_1\gamma_0\gamma_{-1}\ldots\gamma_{-m}$

adică

$$\sum_{i=-m}^{n}(\gamma_i \times 2^i) = \sum_{i=-m}^{n-1}\left((\alpha_i+\beta_i)\times 2^i\right)$$

Suma algebrică în C_2 (3)

- Teorema 2

  dacă suma algebrică a numerelor reprezentate de α şi β nu produce depăşire, atunci reprezentarea rezultatului este

  $$\gamma_{n-1}\gamma_{n-2}\cdots\gamma_1\gamma_0\gamma_{-1}\cdots\gamma_{-m}$$

- Teorema 3

  suma algebrică a numerelor reprezentate de α şi β nu produce depăşire dacă cifrele transport C_{n-1} şi C_n ale rezultatului coincid

Consecințe

- adunarea a două numere în C_2 se poate realiza utilizând un sumator "clasic"
  - biții de semn se adună la fel ca toți ceilalți biți
- testarea depășirii în C_2 se poate realiza adăugând la sumator o poartă NXOR
  - în care intră biții transport C_{n-1} şi C_n
  - nu este deci necesar să fie cunoscute numerele